Course Intro

Hey everyone and welcome to our introductory course on machine learning! Throughout this course, we will learn how what machine learning is and how to use it effectively to solve problems. We will start with an intro to machine learning concepts. This topic can be confusing and the underlying structures are highly complex but we will skip the difficult stuff and dive just deep enough to understand what we are doing and why we take the steps we do when building a machine learning model. Next, we will explore the library **Tensorflow**. Tensorflow allows us to ignore all of the underlying complexities of machine learning and just focus on building and using models. Finally, we will build a **linear regression** model to help plot points on a line. Although this is a very simple model, it will serve as a cornerstone to future, more complex models. There will also be a part 2 of this course available in which we will build a working **digit recognition model using the MNIST dataset**.

Machine learning at its heart can be hard to understand and even harder to implement, so why learn about this topic at all? Why not just use built in machine learning libraries and call it a day? The short answer is that we can more effectively use a tool if we know how it works. Although we don’t need to know how the models work to use them, it is important to understand what is going on behind the scenes as this helps us to approach problems with a better understanding of how to solve them. We will be able to more confidently select the correct tools to more effectively solve a problem and will become better coders in the process. Built-in machine learning libraries are also not very customizable. Learning how to build and train a model yourself will allow you to implement effective custom solutions to your problems. Lastly, Tensorflow actually makes model construction and training quite simple while being highly flexible and customizable, allowing us to implement solutions to almost any problem that can be translated into code.

Intro to Machine Learning

Now that we know what the course is all about, let's learn a bit about the main topic: machine learning. What is machine learning? **Machine learning is the study of statistics and algorithms aimed at performing a task without being explicitly programmed to**. Theoretically, a machine is said to have learned if it produces "better" results over time without modifications to its programming. Practically, this means writing an algorithm, feeding it some data, and letting it interpret the data to find some pattern to solve a problem.

Behind the scenes, machine learning **models consist of layers of connected nodes (often called neurons)**. We will cover model structure in greater detail later but it is important to know now that each node has one or more values assigned to it that, when multiplied with a function of our choice produces some output. Through training, a model can change the values to produce more accurate outputs rather than having us, as the coders, explicitly change the algorithm or values manually. In this way, the model is "learning" as it is improving its results over time using the same algorithm. It should also be noted that some models do not undergo training and are only used to find patterns in data. We will explain the differences in the section common machine learning models.

So what makes machine learning so special? Why not just hard code the algorithms ourselves? The main reason we use machine learning is to help find patterns in data that we wouldn't otherwise be able to see. If we cannot find patterns in data, we cannot hard code algorithms to look for them as we wouldn't know what to tell the algorithm to do. It is this pattern recognition that allows machine learning models to solve recognition, classification, and prediction problems such as speech recognition, image classification, and stock market prediction. Machine learning also helps to customize user experience and tailor solutions to the user based on their previous habits such as with responsive game AIs, health apps, and text suggestion.

What can Machine Learning do – Part 1?

Before we dive into the types of machine learning models out there, let's take a moment to explore how machine learning techniques can be used to solve problems in the domains of **prediction, classification, customization, translation, and AIs.** We won't go too deep into any of these topics yet but this will be a good way to start thinking about how machine learning models may work. After that, we can take a look at specific types of models

- **Prediction:**

  Prediction is trying to guess some future outcome or what some value will be in the future
  Typically based on previous events and the assumption that history will repeat itself so the same thing will happen in the future as has happened in the past
  From a machine learning standpoint, we can teach a model to produce similar values when it sees similar inputs
  If the model is taught to associate input x with output y, if it sees input x again (or something similar), it will know to produce output y
  Example: stock market prediction. Although the stock market is extremely difficult to predict, some models can be used to buy or sell stocks based on price trends. They learn to recognize when the price is low and buy and then sell when the price is high just by looking at the data and how it changes over time. If the model is taught to buy when the price reaches point x as it has seen the price go back up again, it will buy any time the price reaches point x and vice versa.

- **Classification:**

  Classification is grouping inputs into categories based on a predetermined set of features
  This can be easy or difficult depending on how we set things up
  Some features are very difficult to quantify or are very subtle and hard to detect which increases the difficulty whereas some are very obvious which makes things easier
  The difficulty also increases as we add more categories or more features to detect
  Generally, it is easier to make a machine learning model than taking every possible variation on every category into account and having to hard code many, many possible outcomes
  Example: image recognition/classification. Image recognition models will learn to find patterns in pixel values and associate those with certain outputs. Most image recognition is actually image classification, classifying each image into one of n possible categories by allowing the model to make its own associations

-

What can Machine Learning do – Part 2?

In part two of our lessons on understanding what machine learning can do, we're going to explore the remaining three: **customization, translation, and AIs.**

- **Customization:**
  - Customization is providing an experience that is tailored towards you, the user
  - Generally works by learning your preferences (what you have chosen or done in the past) and suggesting or making decisions based on that
  - Many customization models work as prediction models, using what you have done in the past to suggest what you should do in the future
  - Bigger customization systems often have models working on specific aspects and coming together to provide a fully customized system
  - Example: targeted ads. We've all had the experience where we go shopping for something online and then suddenly see ads for those and similar products appear. These ad algorithms take into account what you've been searching and websites you've been browsing to make recommendations for future purchases

- **Translation:**
  - Translation is converting some input directly to some other output, typically as some form of written, spoken, or typed language
  - Translation problems are easy when we are just mapping one input to another output directly and there are a finite number of combinations
  - This problem becomes very difficult when the languages are structured very differently and especially when the addition or restructuring of inputs completely changes the output
  - This is generally solved by introducing some intermediate language that can capture the intent or meaning of the input so that we don't lose any of the meaning in translation
  - Example: language translation. Some languages are structured completely differently so we definitely cannot do a 1:1 mapping. For example, Mandarin and English are completely different languages that don't even share the same alphabet so that makes the problem very challenging. This is solved by capturing the meaning of a sentence to translate in an intermediate language or vector. There is then a decoder that translates the intermediate language into the output language syntax

- **Game AIs:**
  - AI or artificial intelligence is a computer system that makes decisions based on some process of deduction and reasoning
  - Game AIs do this with respect to games, making in-game decisions and reacting based on some sort of intelligent reasoning
  - Older AIs are hard coded with instructions and some step by step algorithm. However, this makes them unresponsive and predictable
  - Newer AIs will respond to player actions and learn how to beat a player by studying what they have done in the past and taking countermeasures
  - Computers can process data very quickly which allows them to assess many scenarios at once and pick the best one (the one that, given the environment and variables, has brought them closer to winning in the past)
  - Example: chess AI. A chess AI is able to, at each move, study all possible moves and understand which move will bring it closer to winning. It knows this based on millions of games played in the past and by learning which combinations of moves and environments has lead it to win in the past. It can assign a value to each move and the one that it takes will be the one with the highest value, or the one that is most likely to cause it to win

Common Types of Machine Learning Models

Now that we have a basic idea of what machine learning is, let's take a look at some common types of models and the problems that they solve. First, we can classify machine learning algorithms into roughly 3 kinds: **supervised, unsupervised, and reinforcement learning**. Below is a breakdown of each as well as some examples of algorithms that you may find in each:

- **Supervised:**

  The model tries to achieve some correct result or target outcome
  The model is fed training data with an input and the correct output for each data point
  During training, the model tries to get as close to the correct output as possible but adjusting its values
  When in use, the model outputs a definite answer (usually some number or list of numbers)
  Often seen in classification and regression type problems
  Some examples: linear regression, decision tree, KNN

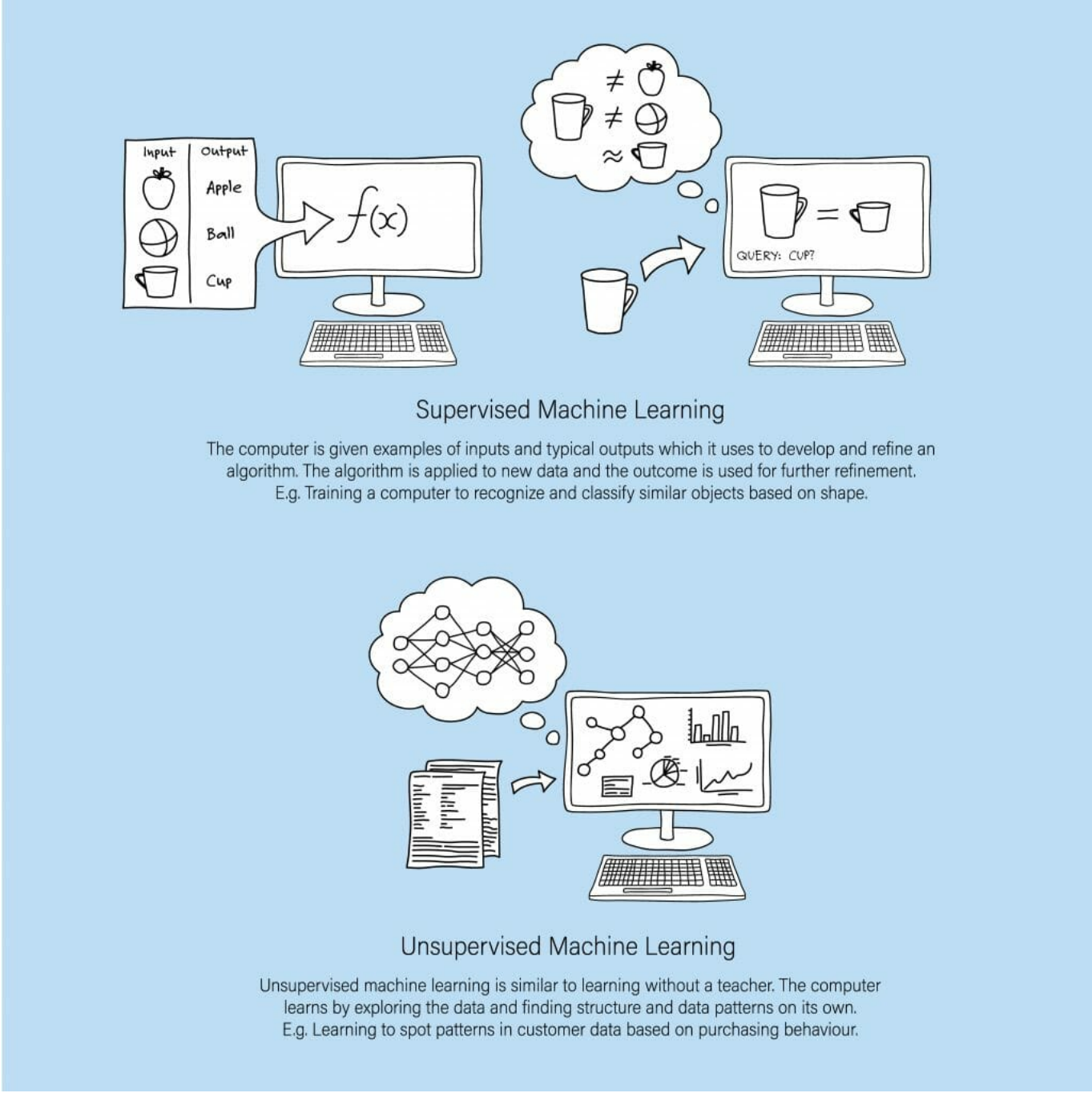
- **Unsupervised:**

  There is no correct answer or target outcome
  The model is fed data and tries to cluster data or find some sort of pattern
  Often seen in data-clustering and analysis programs
  Some examples: Apriori, K-means

- **Reinforcement:**

  The model makes decisions based on past experiences
  The model runs in an environment and learns by trial and error rather than being fed specific data points
  Sometimes it is given a goal or a target it is trying to achieve but is not told how to achieve it
  Often seen in game AIs
  Some examples: Markov Decision Process

We will only be focusing on linear regression in this course and will learn more about it later. The following image outlines the difference between supervised and unsupervised learning:



The above describes categories of algorithms used in machine learning but they do not necessarily dictate the structure of a model. Models that are used to solve conceptually similar problems generally follow the same basic structures. We will go over some common types of machine learning models but, seeing as we will be building only one type of model, let’s get to know that model before exploring the others.

Perceptrons and Other Definitions

Let's now hone in on how our model will be structured while learning some key definitions along the way. Our model will be a **perceptron, or a single layer neural network** which consists of input nodes, one intermediate or hidden layer, and an output layer. A **neural network** is a computational graph that consists of layers of nodes. Data flows through the graph from input nodes to output nodes and is transformed as it goes by various functions. **Each layer in a graph consists of interconnected nodes or a single node** and the structure of the nodes depends on their function. **A node (often called a neuron) is just a point in a graph or neural network through which data will flow and may be transformed.** We will explore specific types of nodes below.

Input nodes are simply there to hold the input values. You should have one node per value. For example, if you are feeding in an image that is 8 pixels x 8 pixels that makes 64 pixels total so you should have 64 input nodes. These nodes pass their values on to the intermediate layer, unaltered.

The **intermediate layer** is the most complex part. It contains layers of nodes that are connected to each of the nodes in the next layer. For example, if there are 8 nodes in the intermediate layer and 8 output nodes, there will be 64 connections between the 2 layers. Each node connection in the intermediate layer has a weight and a bias. **A weight is a value that is multiplied by an input.** A node will maintain a list of the weights for each of the inputs. The weights are modified during training to produce different outputs in the future. **The bias is just a factor added to the product of input and weight and changes the strength of the output.** The bias for each node does not change.

Once we obtain sum of the products of inputs and weights and the bias, we sum all of those to produce the **weighted sum**. This is a combination of all of the inputs, weights, and the bias. We then feed the weighted sum into an **activation function** which typically produces an output of 1 or 0. An output of 1 means that the output is passed on to the next layer. An output of 0 means that the output is not passed on. Different nodes will have different activation functions (we assign the functions) and different functions map inputs to outputs differently. Some common activation functions include Sigmoid, Tanh, and Relu functions. Understanding how the math behind the functions is not important but understanding how each maps inputs to outputs can be. We have summed them up below:

- **Sigmoid:**

  Maps inputs to some value between 0 and 1
  0 means the neuron passes on no output whereas 1 means the neuron passes on the full output
  Becoming less popular due to a few problems such as the vanishing gradient and slow convergence problems

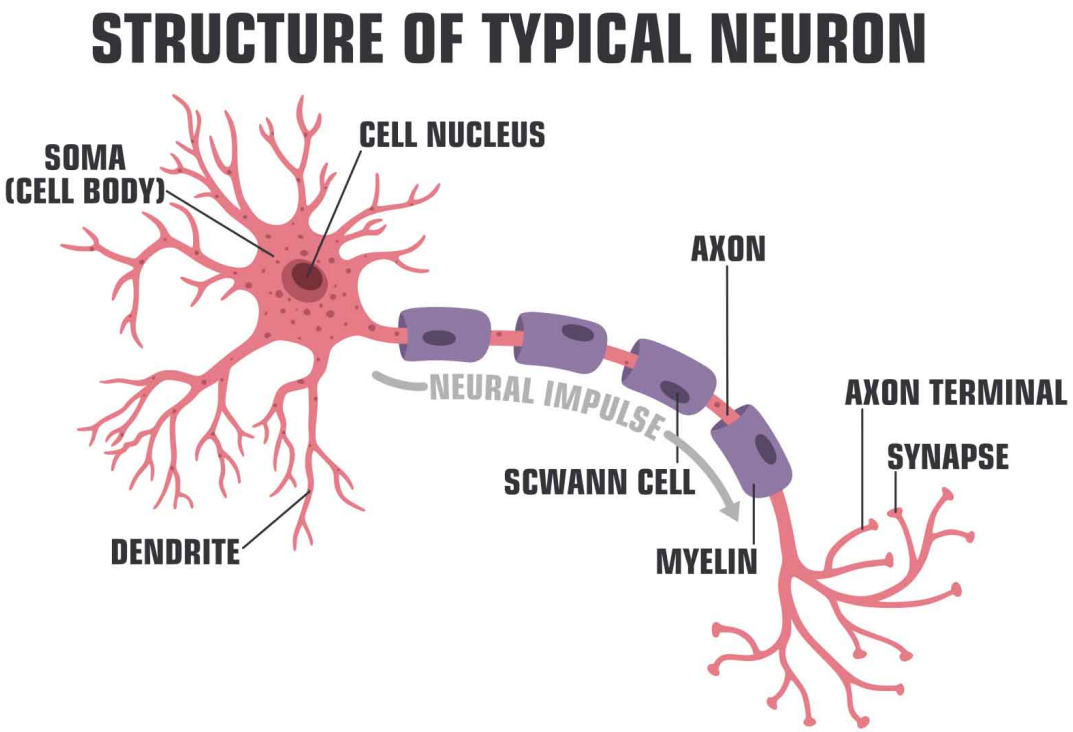
- **Tanh:**

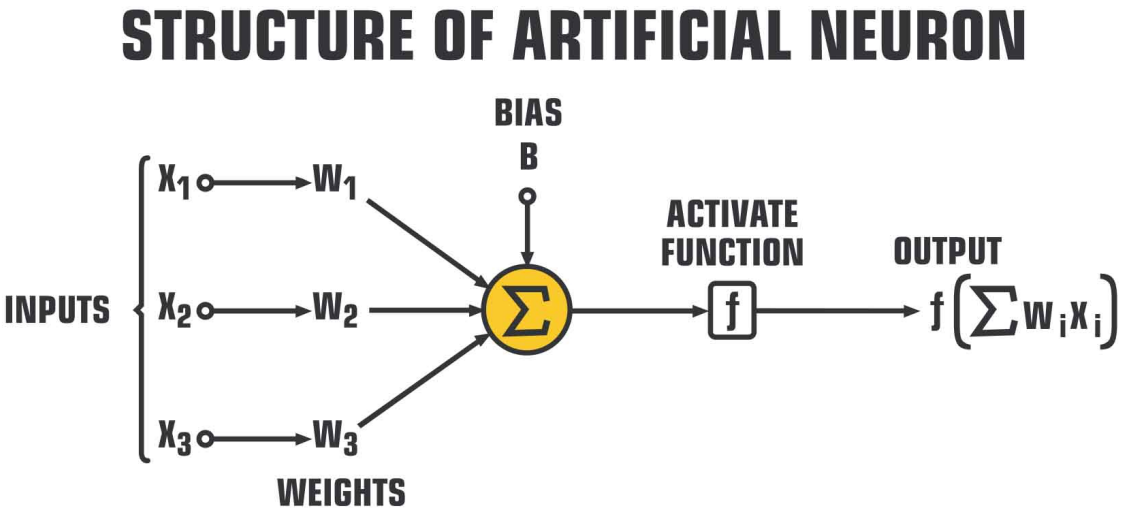
  Maps inputs to some value between -1 and 1
  Better than Sigmoid but still suffers from vanishing gradient problem

- **Relu:**

  Maps inputs to either 0 or the input itself
  If the input is < 0, output is 0 whereas if the input > 0, the output is just the input
  Most popular by far and employed in almost all newer models
  Should only use these in the intermediate or hidden layers

There is also a Softmax function that is used for output nodes in classification problems. It helps compute probabilities that an output belongs to the various categories but isn't so relevant to what we are doing. In fact, we won't be using activation functions at all in our linear regression model. It is a very simple model that only consists of a few nodes strung together. More on that later. This image below helps to demonstrate how inputs into a neuron are combined with weights and biases and fed into an activation function to produce an output:





Other Neural Networks

A perceptron is perhaps the simplest type of neural network and there are many more exciting ones. We will take a quick look at some of the most common types of networks and what roughly speaking how they work. Below are some of the most common types of structures and some examples of where they are used:

- **Feed Forward Neural Network:**

  Data passes through input nodes until it reaches the end of the network
  Fairly simple, not many layers
  Data moves in one direction only
  Typically trained with a **delta rule algorithm** where we calculate the difference between model outputs and expected outputs and try to minimize it by adjusting weights and biases
  Example: facial recognition

- **Radial Basis Function Neural Network:**

  Considers the distance of any point relative to the center
  2 layers
  Employs a **Gaussian activation function** where neurons fire maximally at larger differences between weights
  Typically better at detecting differences or outliers than establishing correlations
  Example: power restoration systems

- **Multilayer perceptron:**

  Every node at each layer is connected to every node at the next layer so it is considered fully connected
  3 or more layers
  Uses **backpropagation** to modify weights and bases. This is essentially a more complex version of the delta rule algorithm. Generally this employs some form of non-linear gradient descent
  Example: classification and prediction models (image classification)

- **Convolutional Neural Network:**

  Similar to multilayer perceptron but at least one of the layers is a convolutional layer

Convolutional layer breaks an image down into many smaller, more abstract images for easier processing
Example: image recognition

- **Recurrent Neural Network:**

Uses **long short term memory or LSTM** cells
Cells contain some memory of previous state to make a prediction for what comes next
Usually not many layers. The few layers run over and over again and self correct through backpropagation
Example: text prediction

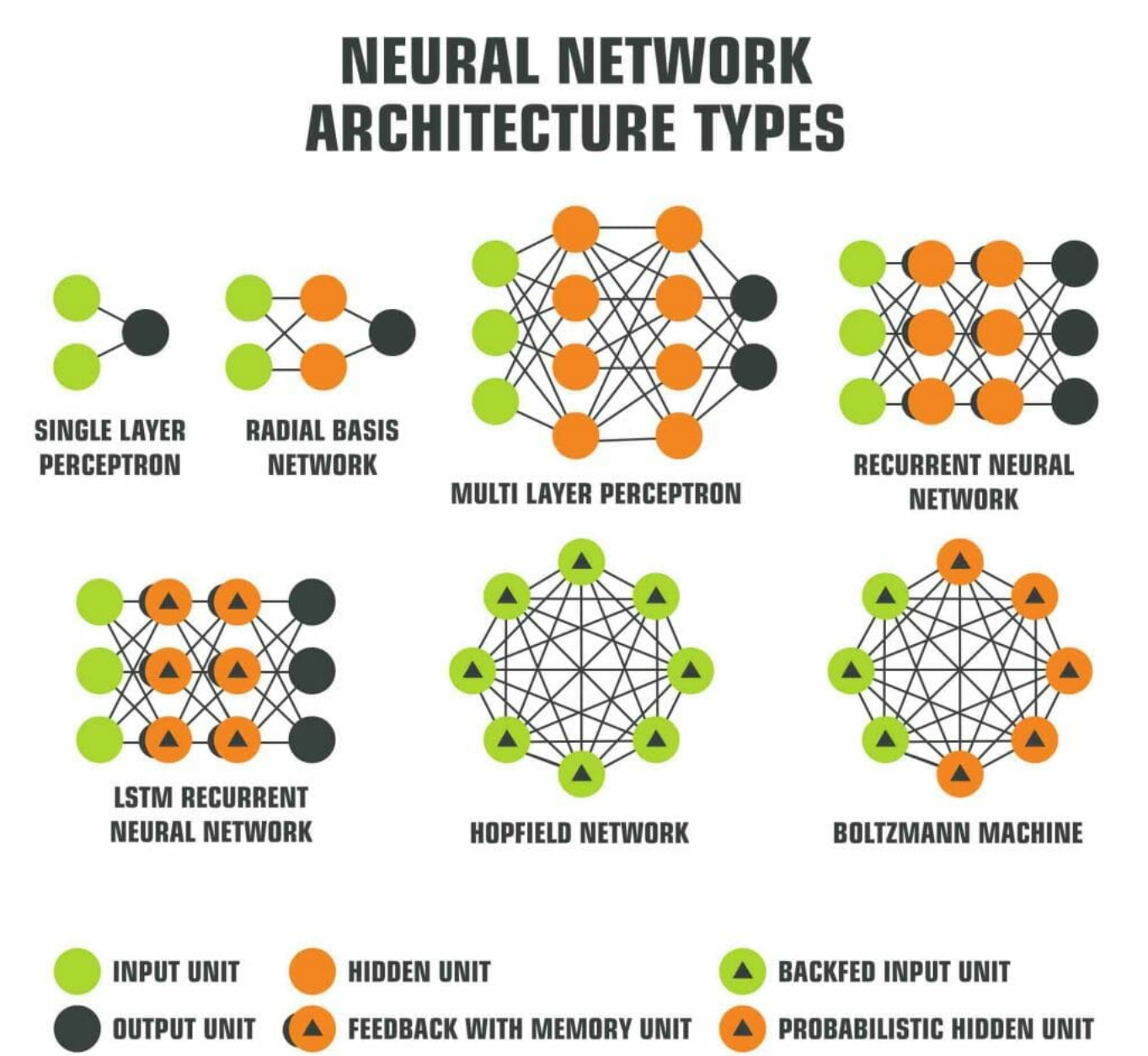
- **Modular Neural Network:**

Uses multiple smaller networks to perform computing similar to computing in parallel
Faster than other neural networks because multiple computations can be performed at once
Also less prone to failure as if one network fails, there are others

- **Sequence to Sequence Models:**

2 networks: encoder to process input and decoder to process output
Encodes input into some intermediate vector
Decoder decodes the output into something meaningful
Very applicable when inputs and outputs are different lengths
Example: chatbots

We won't be building anything complex in this course but we will implement a simple feed forward neural network during our linear regression model construction. It's important that we start at the basics and can then develop more complex models on top. The following image showcases some of the above types of neural networks:



Other Neural Networks

A perceptron is perhaps the simplest type of neural network and there are many more exciting ones. We will take a quick look at some of the most common types of networks and what roughly speaking how they work. Below are some of the most common types of structures and some examples of where they are used:

- **Feed Forward Neural Network:**

  Data passes through input nodes until it reaches the end of the network
  Fairly simple, not many layers
  Data moves in one direction only
  Typically trained with a **delta rule algorithm** where we calculate the difference between model outputs and expected outputs and try to minimize it by adjusting weights and biases
  Example: facial recognition

- **Radial Basis Function Neural Network:**

  Considers the distance of any point relative to the center
  2 layers
  Employs a **Gaussian activation function** where neurons fire maximally at larger differences between weights
  Typically better at detecting differences or outliers than establishing correlations
  Example: power restoration systems

- **Multilayer perceptron:**

  Every node at each layer is connected to every node at the next layer so it is considered fully connected
  3 or more layers
  Uses **backpropagation** to modify weights and bases. This is essentially a more complex version of the delta rule algorithm. Generally this employs some form of non-linear gradient descent
  Example: classification and prediction models (image classification)

- **Convolutional Neural Network:**

  Similar to multilayer perceptron but at least one of the layers is a convolutional layer
  Convolutional layer breaks an image down into many smaller, more abstract images for easier processing
  Example: image recognition

- **Recurrent Neural Network:**

  Uses **long short term memory or LSTM** cells
  Cells contain some memory of previous state to make a prediction for what comes

next
Usually not many layers. The few layers run over and over again and self correct through backpropagation
Example: text prediction

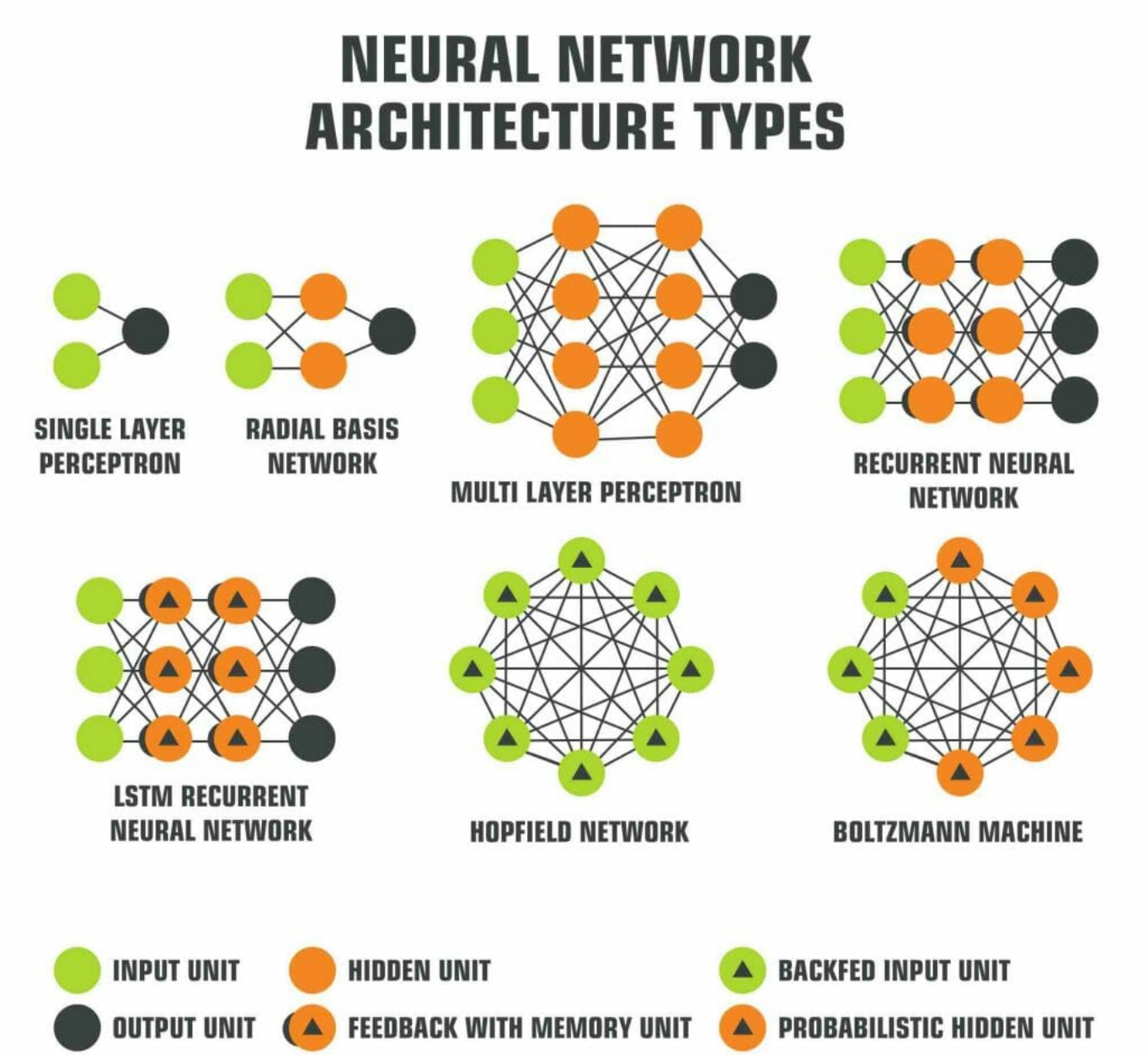
- **Modular Neural Network:**

  Uses multiple smaller networks to perform computing similar to computing in parallel
  Faster than other neural networks because multiple computations can be performed at once
  Also less prone to failure as if one network fails, there are others

- **Sequence to Sequence Models:**

  2 networks: encoder to process input and decoder to process output
  Encodes input into some intermediate vector
  Decoder decodes the output into something meaningful
  Very applicable when inputs and outputs are different lengths
  Example: chatbots

We won’t be building anything complex in this course but we will implement a simple feed forward neural network during our linear regression model construction. It’s important that we start at the basics and can then develop more complex models on top. The following image showcases some of the above types of neural networks:



Steps to Building a Machine Learning Model

By now, we should understand the typical structure of a machine learning model. However, building the model is just one step out of 3 or 4 steps, depending on the type of algorithm we decide to use. Strangely enough, the model construction is sometimes the quickest step in the process! Additionally, we have to gather and format data, train the model (for supervised and reinforcement learning algorithms), and test the model. Below, we will outline each step in greater detail.

Before we start gathering data or building a model, **we have to figure out whether or not our problem requires machine learning to solve or can even be solved with machine learning, for that matter.** Not all problems require machine learning to solve. Once we determine that we do need machine learning, **we need to figure out which type of algorithm to use**. If we are just looking for patterns, we should choose an unsupervised algorithm. If we are looking to predict future values or categorize data, we should choose a supervised algorithm. If we want our model to learn some behaviour, we should go with a reinforcement algorithm. Within each of these categories, we choose a neural network structure that will best solve our problem. Selecting the optimal structure can be a difficult process and the correct answer is not always clear. For example, text-related problems such as text prediction and summarization are often solved through recurrent neural networks. However, some solutions employ encoder-decoder type networks instead or have a combination of both. It may take you a few tries before you find the best solution.

A good place to start is to look at models that solve similar problems and see if we can implement something similar. For example, if we want an image classifier, we should look at how other image classifiers are built, what structure and algorithms they use. Of course we should never copy someone else's work, but looking at similar solutions may give us a starting point.

Gathering data:

Once we have an algorithm type in mind and a rough idea of how we will structure the neural network, **we need to gather data that will help us to train and test the model.** The kind of data we need depends on the problem we are solving as well as which algorithm type we are using. For example, with reinforcement learning, we don't necessarily gather outsider data; we analyze the data points that the algorithm produces while it runs. With unsupervised learning, there is no training at all so the data we gather is purely for analysis. With supervised learning, we need to gather enough data that we can train and test our model. As we are only dealing with supervised learning in this course, this will be our main focus.

When gathering the data, we want as much of it as possible. The more data we have, the more exposed our algorithm is to any possible input-output combination and it should be able to respond more appropriately to something it has seen before. We want varied data that will cover every scenario we can think of but it should all be formatted similarly. For example, if we are classifying images, we want all of the images to be the same size and all be color or all be black and white. If the data isn't all the same, we need to format it. For example, if we have some color images and some black and white then it would be best to grey-scale all of the images to just deal with black and white images.

Once we have all of the inputs, we need to label them with the correct output. It is up to us how we want to interpret the labels. Sometimes a label is simply one correct output value as with a prediction. Sometimes, especially with categorical data, we have a list of numbers that represents the probability of belonging to a category. For example, if we are classifying images into one of ten categories, we may assign each image a label like this:

[0 0 0 0 0 0 0 0 0 1]

This indicates that the image belongs to the final category and not any of the others. This particular

format is called one-hot encoding and we will see this a lot with categorical data. It is important that we keep our label format consistent and can understand the results.

Once we have a label for each input, we divide our data into training and testing data sets. We want unique data points in each as using the same data points for training and testing can give us false confidence in our test results due to the algorithm already having seen that data point. The idea behind training is that we expose the model to as many different inputs as possible so that it knows how to respond appropriately and map it to the correct output, or as close as possible. Generally we do an 80-20 split of training to testing data (80% of the data is used in training and 20% in testing) although there is no hard and fast rule about this.

There are thousands, if not millions, of datasets available online for all sorts of problems from image classification to text prediction. Many of these datasets are pre-formatted to all be consistent, labelled, and even split into training and testing sets. Some of them are free, others we have to pay for. If you don’t want to pay to access the data sets or data is not available, you will have to gather your own data which can be a painstaking process. Some common techniques involve going out into the field to gather data, conducting surveys, or scraping data from web pages or retrieving it through APIs.

Building the Model

Once we have the data, we need to build the model itself. **The model is often called a computational graph as it is a network of nodes that performs some kind of computation.** This can be simple or complex depending on the type of problem we are trying to solve but it always starts with input nodes and ends with output nodes. We need to make sure that there is one input node for each input value and one output node for each label value. For example, if we are inputting 8 x 8 images and outputting a categorical label of ten categories we need 64 input nodes and 10 output nodes.

When building the model, especially with a fairly low-level library like Tensorflow, we have options to customize each node and layer as we see fit. Within the nodes, we might want to specify initial weights, biases, and activation functions. Within each layer, we want to specify how many nodes we have, what kind of inputs and outputs to expect, and any operations that need to be performed. When putting layers together, we want to start with the input, build any hidden or intermediate layers, and finish with the output layer. Always make sure that each layer is getting the correct input shape and that each node is connected to each of the nodes in the next layer.

Training and Testing

Once we have the data and the model is built, it is time to actually run it! **With supervised learning, we start by training it until we reach an acceptable level of accuracy and then test the model.** If the tests produce the desired accuracy then we can move on, otherwise we refactor the graph, gather more data, and repeat the process. It's important to note that unsupervised algorithms require no training and reinforcement learning algorithms train by learning behaviours rather than trying to map inputs to outputs.

Training is the process by which the model adjusts its weights to produce more accurate results. The details of this process are dependent on the structure of the graph and the type of algorithm but many, including the linear regression algorithm, follow the principle of backpropagation. To understand backpropagation we need to know a couple of other terms: loss and optimizer. **Loss is the total difference between the correct and incorrect answers.** We are trying to minimize this as the smaller the number, the closer the output is to the correct answer. **That is the job of the optimizer which is another function that alters the weights within each node to hopefully produce better outputs and minimize loss.** There are many optimizer functions but we will probably stick with a **Gradient Descent optimizer.**

Let's look at a quick example. Let's say we have 5 data points whose outputs should be 1, 2, 3, 4, and 5. During the first round of training, our algorithm outputs 1.5, 2.3, 3.9, 3.7, and 5.1. We could calculate the loss as the square root of the differences squared so:

(1.5 – 1)^2 +

(2.3 – 2)^2 +

(3.9 – 3)^2 +

(3.7 – 4)^2 +

(5.1 – 5)^2 =

1.25

Here we get a total loss of 1.25. We want loss to be as close to 0 as possible so the optimizer will see the result, see that there is loss, and go back through the nodes and change the weights to hopefully

produce better results. If it changes the weights one way and notices that loss is decreasing, it will continue to alter them that way. If it notices that loss is increasing, it will change them in a different way.

Of course, there won't be much change if we only run through the training data once or twice so we usually run through the training data many many times. **The number of times we run through the data is called the number of epochs. Generally speaking, the more we run the data, the better the results.** We may run through the data hundreds or even thousands of times so that a tiny modification in the weights with each run and sum to a big difference in the weights and loss over the long term.

Once we run through the data many times, we will get some sort of accuracy reading which will be the percent of times the model got the correct output for a given input, or at least we are happy with a sufficiently small loss. **Once we are happy with the results, we run the model on the test data set.** This does not alter the node weights like training does; it is essentially a simulation of how well it would perform in the real world when faced with new data it hasn't seen before. **If we are happy with how well the tests go, we can go ahead and publish the model and call it a day! If we are unhappy with the results, we should refactor.** Refactoring could involve changing the model in some way like changing the layer structure, changing the node structure, or perhaps even choosing a completely different structure or algorithm. It could also just be a matter of obtaining more data and training the model further.

Intro to Tensorflow

Now we know the basics of how machine learning works, let's see how we can actually implement this using Tensorflow. A good place to start would be learning about what Tensorflow is! **Tensorflow is an open source library that provides tools for statistical analysis and machine learning.** It contains everything we would need to create a machine learning model and train and test it. It is readily available through most Python platforms and can be easily installed through Anaconda to be used in Jupyter Notebooks.

Tensorflow provides an easy to use layer between the gritty details of machine learning mechanisms and model construction. Rather than implementing the algorithms, functions, and node objects ourselves, we can create layers using various node and operations classes. Tensorflow gives access to pre-written activation functions, optimizer functions, and methods to help us train and test our models. In short, it provides us with all of the tools we need to build complete models. Before libraries like Tensorflow, we would have to program all of that in ourselves which is difficult without a formal education in machine learning techniques and matrix algebra.

Although it makes building machine learning programs much easier, Tensorflow is still considered fairly low-level however, and there are libraries built on top of it that make the process even easier. One which we may explore in later lectures is Keras. **Libraries like Keras provide higher level functionality that makes our lives easier but they are less customizable.** It is a good idea to build a solid foundation with Tensorflow before moving onto something like Keras.

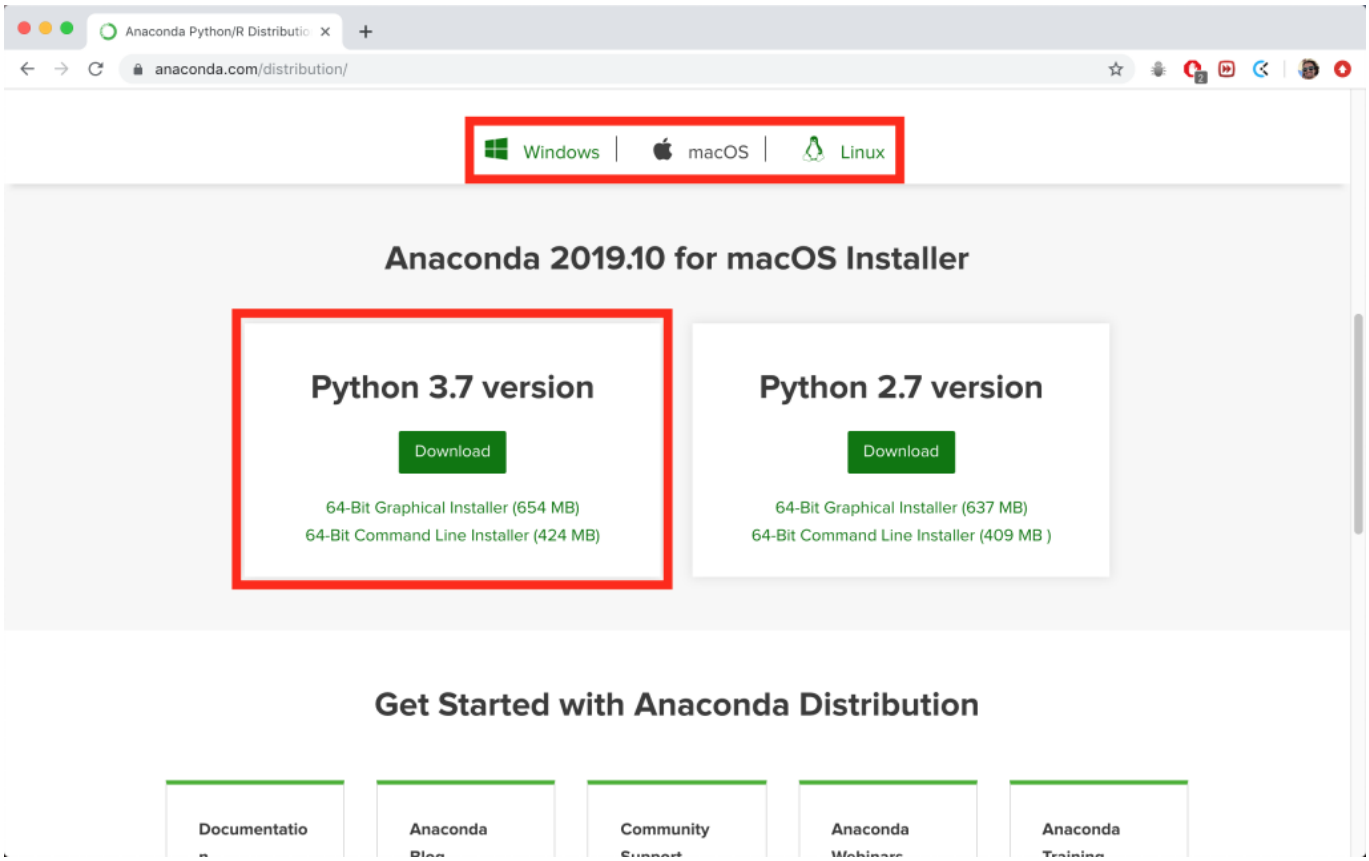
Tensorflow is a massive library that covers provides tools for pretty much every aspect of machine learning so we won't have time to go through it all. Instead, we will focus on some fundamental components of Tensorflow and how we use them to either build a model or train or test the model. A good place to start would be to install Tensorflow and add it to your development environment. Let's start by installing Anaconda as we will be using that to write and run our code.

Installing Anaconda

Anaconda is a development environment that contains all of the tools to write and run Python code. It is quite a large app so make sure that you have plenty of space in your system. Alternatively, if you really don't like using Anaconda, you can write and run the same code with any text editor and terminal or Python-enable IDE. You will want to follow this link for installation steps:

https://www.anaconda.com/distribution/

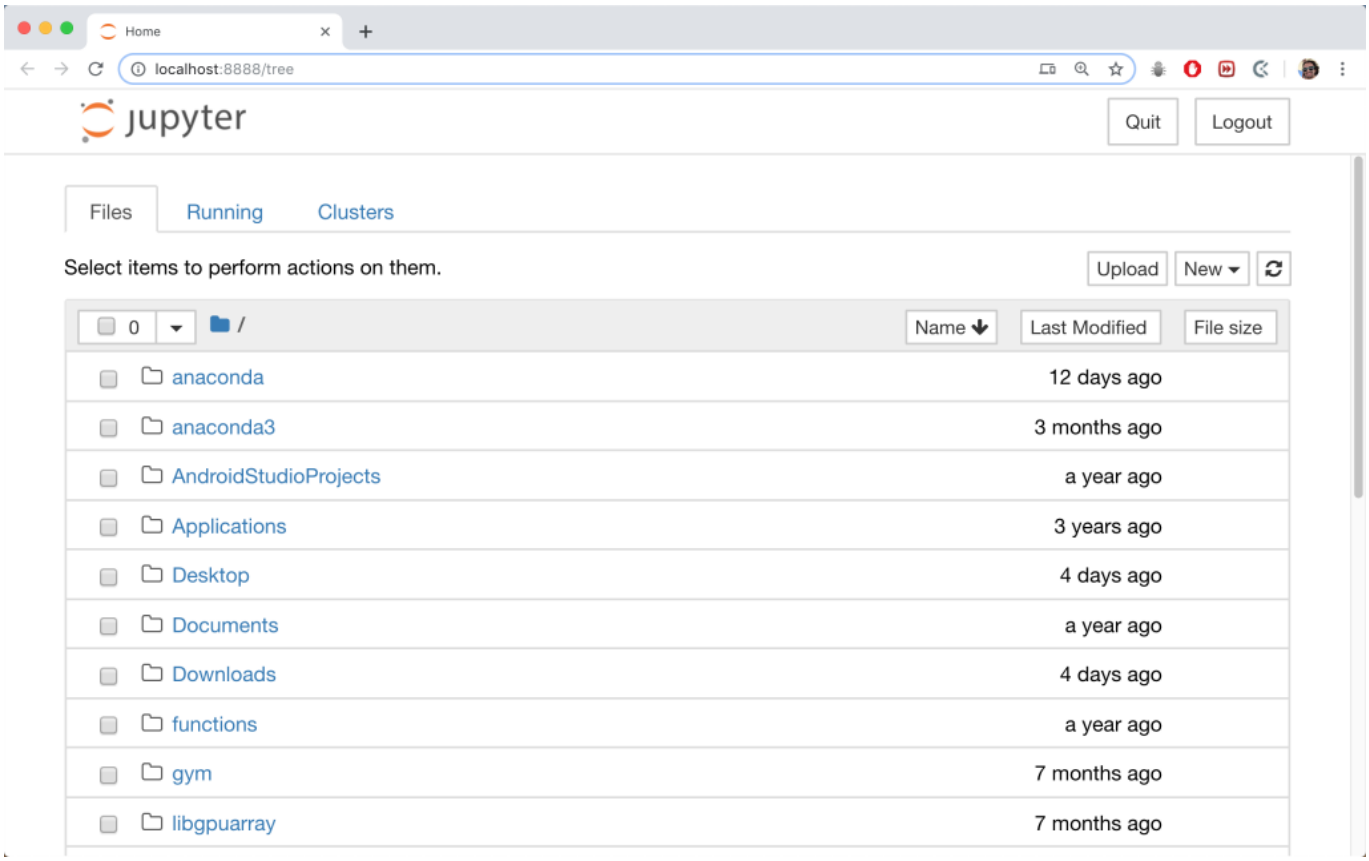
The page will look like this:



Scroll down until you find the "Installing on x" where x is your system and click on the appropriate link. Select the latest version of Python (at the time of writing this tutorial it is 3.7), download the correct installer, and install Anaconda.

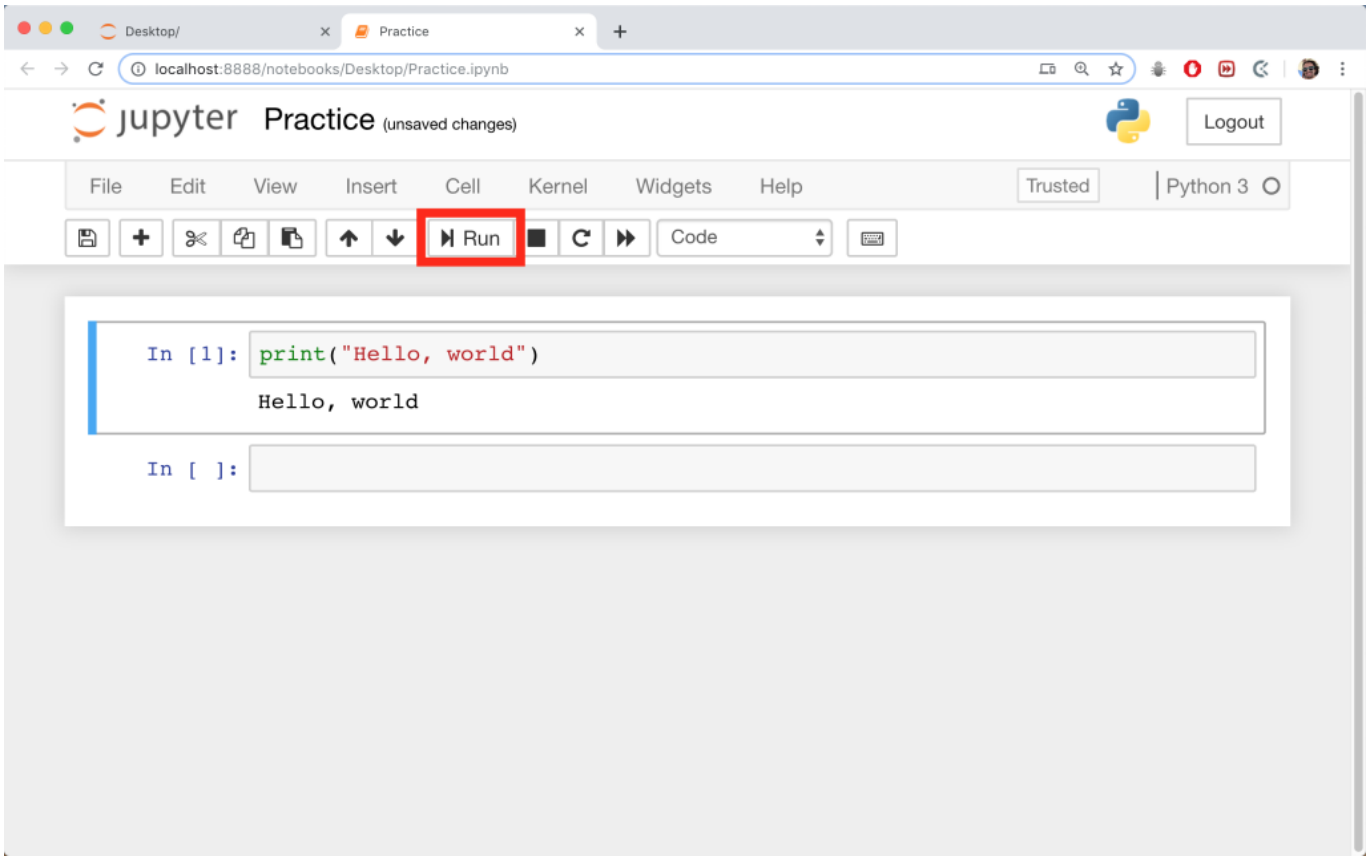
NOTE: Make sure you have a Python 3 version (3.7 or higher) on your Anaconda, not the 2.7 version, as it does not have some key features we're going to use during this course. If you have Anaconda with a 2.7 version of Python already installed in your system, we would recommend uninstalling it and downloading/installing a newer version of Anaconda with Python 3 in turn.

Once that is done, you will have Anaconda installed so start the program. **We will be working in Jupyter Notebooks so go ahead and start one from the Anaconda Navigator.** This should start a terminal/cmd prompt session and open a file explorer in your default browser. Although yours will be different depending on your file system, my default Jupyter Notebook start up looks something like this:

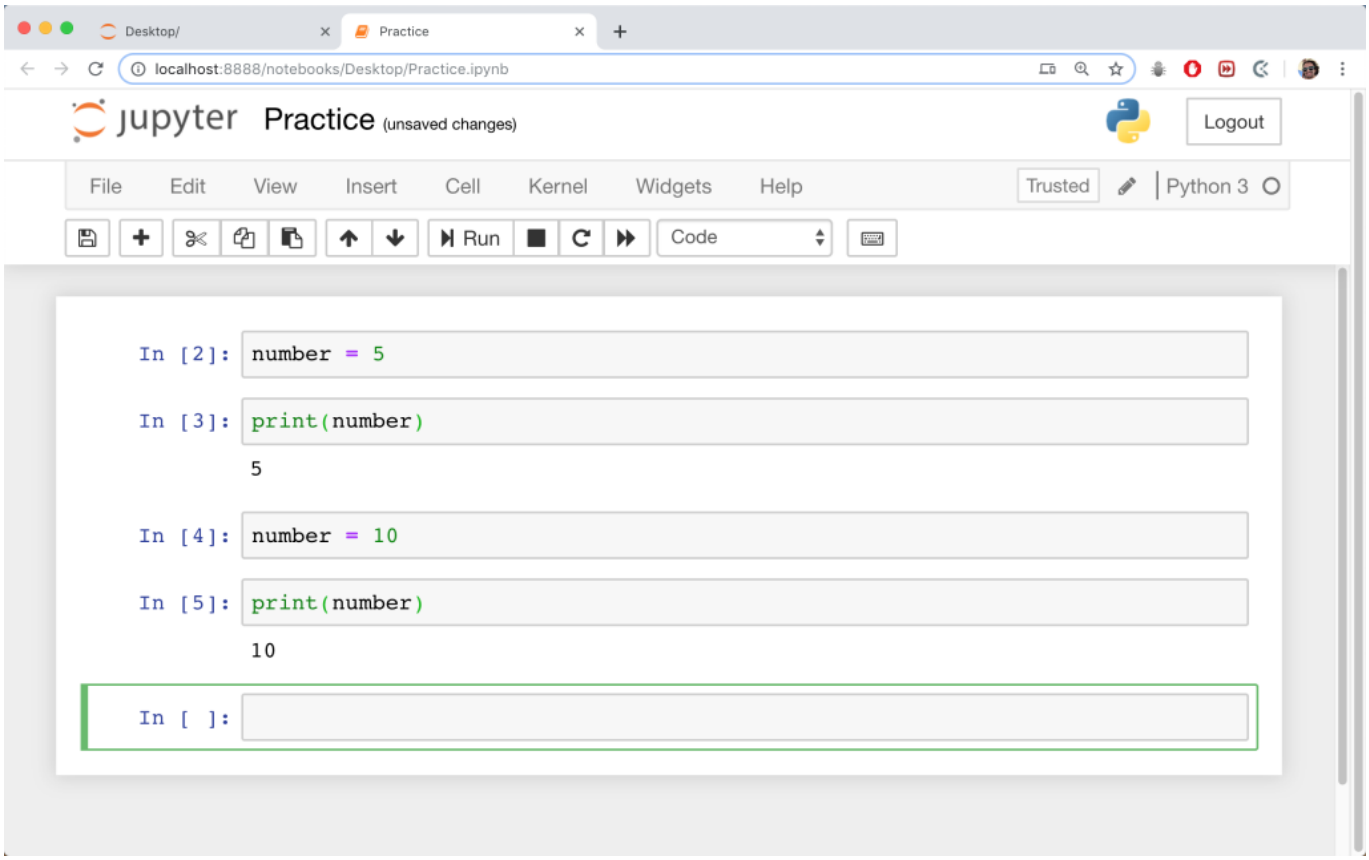


Navigate to where you want your file to live and start a new notebook by clicking the “New” button in the upper right corner and selecting a Python 3 notebook.

Once you have a Python 3 notebook started, you should start by giving it a name. Jupyter notebooks run code one cell at a time. If you write code in a cell, you can run the cell by selecting it and clicking the “Run” button like this:



As you can see, it runs the cell and shows the output. It’s important to know that any values stored will persist over time. This means that if you write a variable in one cell and change its value in another, that changed value will persist throughout the program. An example can be seen here:



I have 4 separate cells and ran them in order (2 to 5). Even though cells 3, 4, and 5 didn't have number declared in them, they remembered the state of that variable. This can cause problems later down the line if you do not keep track of how you are changing variables because in the above scenario, if I re-ran cell 2 (number = 5), the value 5 is now stored in number even if I create and run more cells after cell 5. Make sure you keep track of the order in which you are running your cells (the numbers to the left do that for you).

The code in the video for this particular lesson has been updated, please see the lesson notes below for the corrected code.

Now that we have Anaconda installed and know how to use Jupyter Notebooks, we can install Tensorflow. To install Tensorflow specifically for Anaconda, open a terminal/cmd prompt and if you're using a **mac or linux**, run:

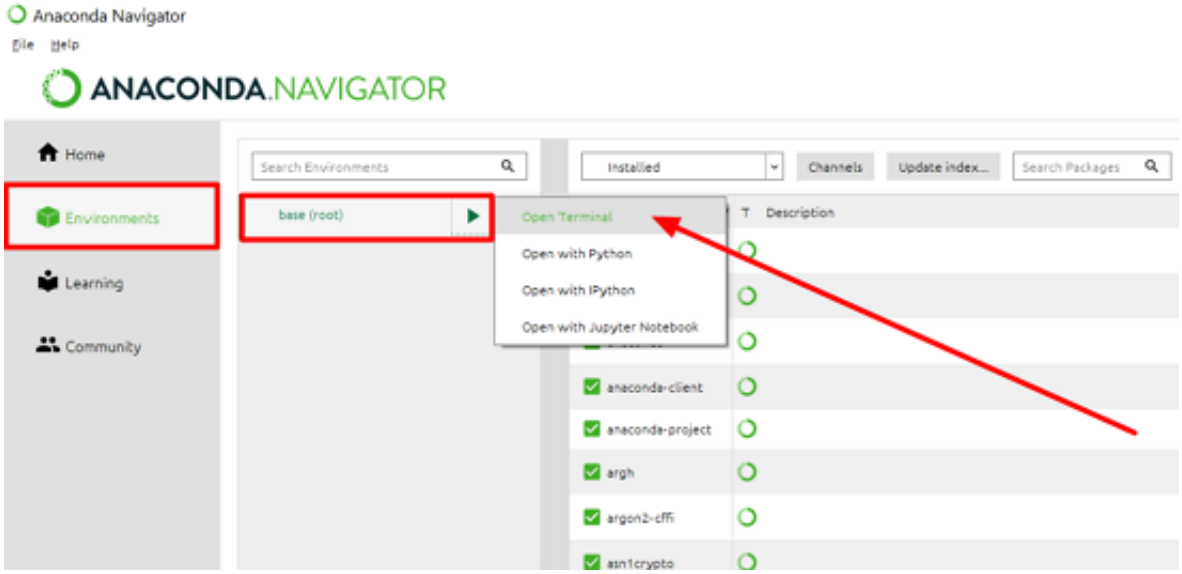
```
sudo conda install tensorflow
```

To install Tensorflow through **pip** for use in other environments, for **mac and linux** run:

```
sudo pip install tensorflow
```

The following code and instructions have been updated, and differ from the video:

If you're on **Windows**, **open the Anaconda terminal** as follows:



Go to the '**Enviroments**' **tab** then **click** on the '**base** (root)' option and '**Open terminal**'.

Next, **run**:

```
pip install tensorflow
```

The following code and instructions have been updated, and differ from the video:

Note that this course uses **Tensorflow 2.0 or higher**. You can **check** your **Tensorflow version** by **running** this on the **Anaconda terminal**:

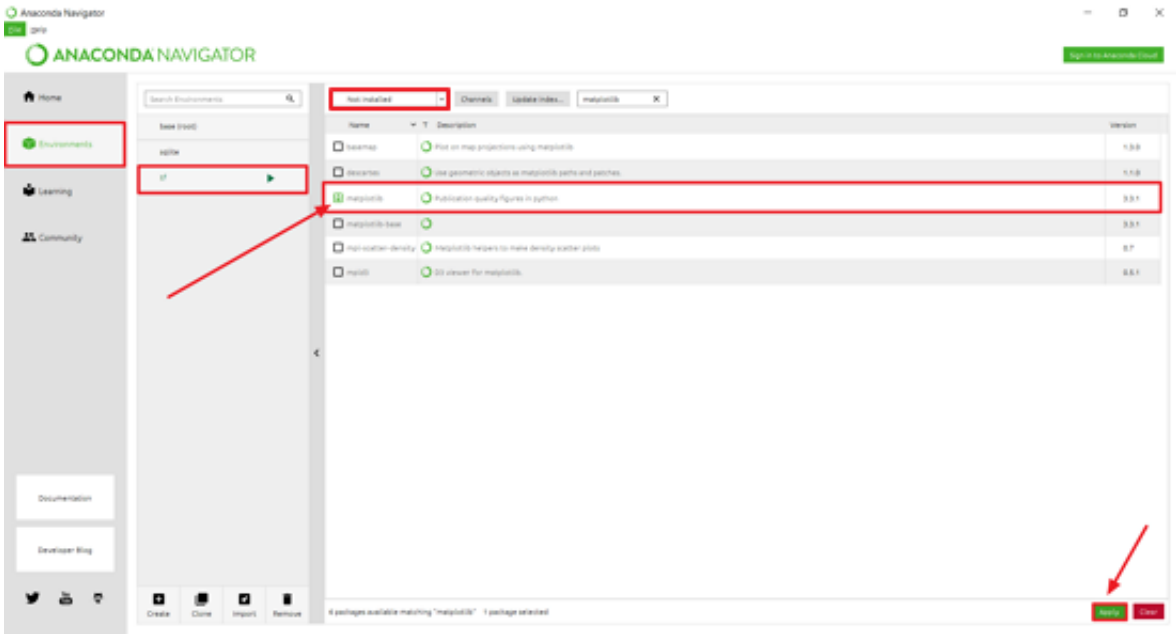
```
pip show tensorflow
```


In case you need to **upgrade** your Tensorflow to its more recent version, you can do so as follows:

```
pip install --upgrade tensorflow
```

Note: If you run across an error due to the current Anaconda Navigator having Python version 3.8 or similar just follow the instructions seen here https://docs.anaconda.com/anaconda/user-guide/tasks/tensorflow/

If you **install Tensorflow** with the **link** above (that is, by creating a new environment), you'll need to **manually install matplotlib** in your newly created **tf environment** by going to the '**Environments**' **tab, selecting** 'tf' in the environments menu and **searching** for '**matplotlib**' in the search box with the **drop-down filter set to 'Not installed'** or 'All':



Importing Tensorflow

Now that we have TensorFlow installed, just **add this line to your Jupyter notebook** to access it:

```
import tensorflow as tf
```


Linear Regression

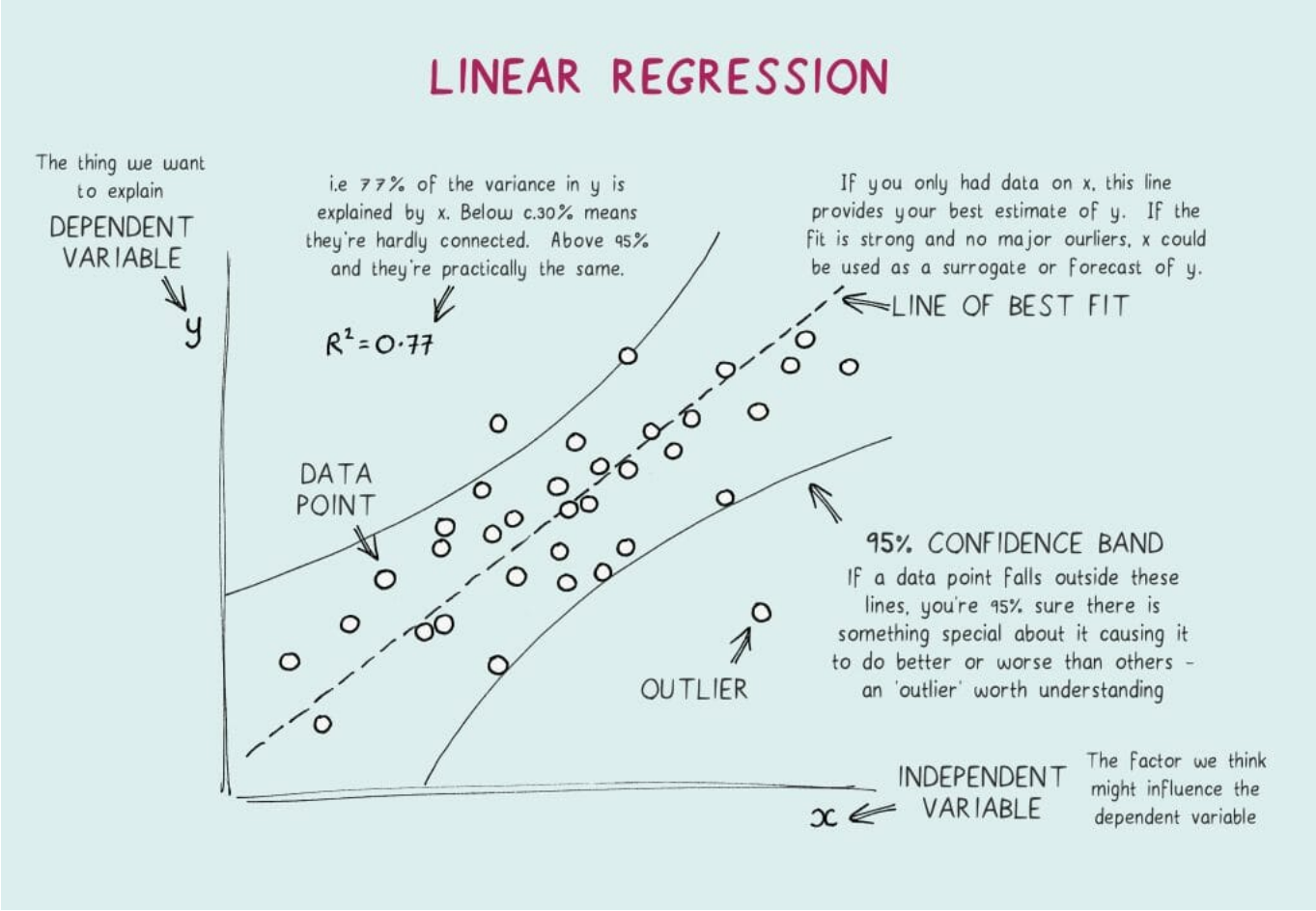
Before we get to know Tensorflow, we should take a minute to learn about Linear Regression. After all, our model will try to implement a solution to this problem. **Linear regression is fitting a line through data in such a way that minimizes the average distance between all points and the line.** Essentially, it is a line of best fit. We use machine learning to solve this by adjusting the slope and intercept of a line through training.

Every line can be described by this equation:

```
y = mx + b
```

Where y is the output or graph y value, m is the slope or rate of change, x is the input or graph x value, and b is the y intercept or the point at which the line crosses the y axis. Inputs and outputs will be determined by the data that we feed into the model. The slope and the y intercept will be adjusted to describe the line of best fit. During training, we feed in inputs as x with the corresponding outputs or labels as y. We usually start out with some arbitrary values for m and b which get better over time to produce more accurate results.

With each step during training, we produce a loss value, or the sum of the differences between model outputs and the labels. These can be called actual output and expected output, respectively. We get the loss by drawing the line and finding the difference between all points' y values and the y value on the line for that particular x, given the current m and b values. **Our goal over training is to adjust the m and b values to minimize the loss.** It adjusts the line to get as close to the line of best fit as possible. We can then use that line to predict any other value, or get as close as possible. The following image outlines what linear regression looks like:



Now that we can access Tensorflow and understand the problem we are trying to solve, let's explore how to actually build and run a model. We will explore the Tensorflow library while we implement our linear regression model. After this, we will train and test the model with some fake data. By the end, we will have a complete linear regression model.

A good place to start here would be to get some data. We don't have an actual problem to solve just yet so let's create some arbitrary fake data and try to fit a line through it. A good way to do this is to randomly create some inputs and outputs. **We can do that with numpy.linspace.** This is a way to create an array of n data points between a low point and a high point. If we wanted to create, say 100 points between 0 and 4, we could do so like this:

```
import numpy as np
x = np.linspace(0,4,100)
```

We need some labels for this and we can do so by feeding them into a y = mx + b equation. We should arbitrarily assign values for m and b. These can be whatever we want so for example:

```
m = 2
b = 0.5
```

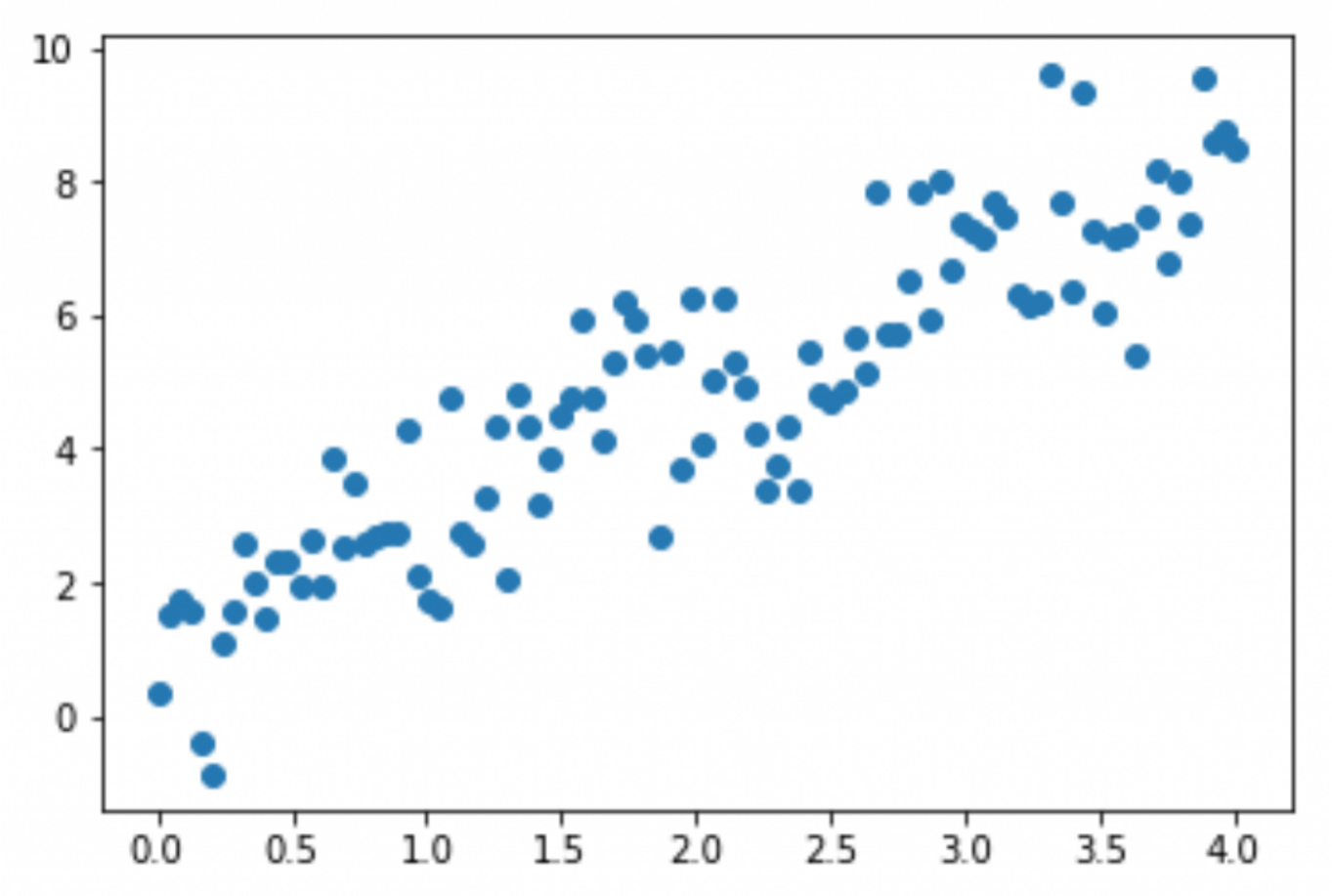
This would create a line with a slope of 2 and a y intercept of 0.5. We want to get the labels but also want to introduce some noise so we will add some random variance. Something like this will do:

```
y = m * x + b + np.random.randn(*x.shape) + 0.25
```

If we want to see what it looks like, we can import pyplot and plot the data like this:

```
import matplotlib.pyplot as plt
plt.scatter(x,y)
```

To get an output something like this (it will vary based on the random values generated):



Good stuff! So we have some inputs and labels. Now we can start coding with Tensorflow specifically. A good place to start would be a Variable node. **A tensorflow variable node provides a way to store a value or values that can change over time.** It can store a lot more information such as the shape of the data, the data type, a name that we can assign, etc. For a complete list of parameters, check this link out:

https://www.tensorflow.org/api_docs/python/tf/Variable

We're not interested in all of the parameters, we only really care about the initial value for now. A good place for a Variable node would be a weight or a bias as these values will need to be initialized and then changed during training. We can set them up like this:

```
weight = tf.Variable(10.0)
bias = tf.Variable(10.0)
```

When we want to change them, we call the function assign_sub(new_value) like this:

```
weight.assign_sub(5.0)
```

We won't do that just yet, though. **As an aside, if we want to specify a data type, we should use Tensorflow's built-in data types.** Some examples are tf.string, tf.float32, etc. For a complete list, check out this link below:

https://www.tensorflow.org/api_docs/python/tf/dtypes/DType

The data type is inferred from the values we assign so this isn't necessary here. **The next step is to implement a way for the model to actually calculate an output based on the current weights and biases.** That's quite simple for us; we just need to replicate y = mx + b. However, it's easier if we can store all of these values in an object so that we can keep track of them later on in our program. We can set the weight and bias when we initialize the model object and have a function to produce outputs given the current weight, bias, and some input x. The model class could look something like this:

```
class Model:
    def __init__(self):
        self.weight = tf.Variable(10.0)
        self.bias = tf.Variable(10.0)

    def __call__(self, x):
        return self.weight * x + self.bias
```

We named the function “call” for a specific reason which we will explore in a minute. **The next step in the process is to build a loss function.** We will need this to get the difference between the model output (actual output) and the labels (expected output). It’s not just as simple as subtracting one from the other; we have to get the absolute value (because some points will appear above the line and some below). **We then have to get the average across all points.** We can do that with two functions: tf.square (to turn all values positive) and tf.reduce_mean (to get the average of the square roots). We can write a function to do that by taking in the actual and expected y values like this:

```
def calculate_loss(y_actual, y_output):
    return tf.reduce_mean(tf.square(y_actual - y_output))
```


This ensures that we get the average difference between all outputs. **Now we know what value we are trying to minimize and can build a train step function.** This function needs to take in the model (to get the current weight and bias values), the current x and y values, and a learning rate. The learning rate dictates the rate at which the weights and biases will be adjusted during training. The higher the learning rate, the faster values will change but the less accurate the model will be. With a lower learning rate, we can be more accurate but the model takes longer to train.

Within this function, **we need to initialize tf.GradientTape() to get access to the gradient descent function, get the current loss, add the gradient descent function, get the results of changing the weights and biases, and then store the new values back in the model.** We can achieve it with this code:

```
def train(model, x, y, learning_rate):
    with tf.GradientTape() as gt:
        y_output = model(x)
        loss = calculate_loss(y, y_output)

    new_weight, new_bias = gt.gradient(loss, [model.weight, model.bias])
    model.weight.assign_sub(new_weight * learning_rate)
    model.bias.assign_sub(new_bias * learning_rate)
```

The third line (y_output = model(x)) is the result of model.__call__. The “call” syntax is a special function that allows us to omit the name of the method when calling it. We feed the loss and the current weight and bias into the gradient function to get the derivatives for such weight and bias. We then multiply them by the learning rate to allow for extra accuracy and feed the adjusted values back into the model. Every time assign_sub is called, it will adjust its own weights and biases in such a way to minimize the loss.

This function will be called at every iteration of our training loop. Speaking of which, it is time to add the training loop. This will be a simple for-loop. We specify the number of times the loop will run as the number of “epochs”. Generally speaking, we get better results with a higher number of epochs as the model has more time to examine the data. However, past a certain point, we start to see an insignificant change in accuracy or loss as running through the same data can only do so much. Also, the more times we run through the training process, the longer the model takes to train so we don’t want to set the value too high. 100 should be fine. We can start things off like this:

```
model = Model()
epochs = 100
learning_rate = 0.15
```

A learning rate of 0.15 is fine as it is not too high to see a decrease in accuracy but not so low as to see insignificant changes. Now it’s time to run the loop. With each iteration, we want to print out the current loss value as well as run the train step. We can achieve that with this code:

```
for epoch in range(epochs):
    y_output = model(x)
    loss = calculate_loss(y, y_output)
    print(f"Epoch: {epoch}, loss: {loss.numpy()}")
    train(model, x, y, learning_rate)
```

This runs the train function and so adjusts the model weights with each epoch. It also calculates the current loss based on the current weight and bias and prints it out. At the end, the model will have adjusted its weights and biases to the point where it can produce pretty good results and we should be able to draw a line of best fit through the data. Run this loop and you should see 100 print statements describing how the loss changes. It should decrease very rapidly at first and then slow down, starting at around 730 and ending close to 1. We're not so interested in achieving an accuracy measurement with this model; we just want to minimize loss so that we can draw a line through the data and use that to predict future values.

If you want to know the final values of weight and bias, print them out like this:

```
print(model.weight.numpy())
print(model.bias.numpy())
```

These values should be stored in your model object after training. They should either be very close to your original values of m and b, respectively, or at least describe a line that makes sense. In fact, you can draw a line through your original data set by doing this:

```
new_x = np.linspace(0,4,50)
new_y = model.weight.numpy() * new_x + model.bias.numpy()
plt.scatter(new_x,new_y)
plt.scatter(x,y)
```

This creates 50 new inputs and 50 new outputs where the outputs are the results of using the model's final weight and bias as m and b. We should see that even though the final weight and bias may not be the same as m and b, they do provide a good line of best fit through the data.

And that's it! We have successfully built our simple linear regression model using Tensorflow and employing perceptron logic to a self-made data set. Although it does nothing fancy, it serves as a benchmark to building a more sophisticated model and taught us some machine learning concepts along the way.

Summary

Congratulations on reaching the end of this course! Throughout, we learned what machine learning is and how to use it to build a simple feed-forward neural network. We started with an intro to machine learning then moved on to learning how we can solve various problems by applying machine learning techniques. We then learned about some types of models and how they excel and solving specific types of problems. We installed the necessary software, learned about linear regression, then built our linear regression model from scratch. Finally, we trained and tested the model and saw that even with a very simple single layer perceptron, we can achieve pretty good results!

From here, the next step would be improving the model. Try tweaking the initial variable values, the initial data, and the learning rate or number of epochs and see how each has an effect on the final result. You could try implementing a more sophisticated model, although that may take a greater understanding of Tensorflow. Now that you understand general machine learning concepts, you can begin to explore other problems and the solutions to those problems. It’s a good idea to start with an area that you’re interested in, such as image recognition, or text summarization and focusing on that for a while. Definitely check out some other open source projects and contribute to them if you’re feeling bold.

I hope you enjoyed our course and learned a lot from it! We have follow up courses on specific aspects of machine learning such as image recognition so stay tuned for those. Otherwise thanks very much for taking this course and hopefully, we will see you in another!

In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

You can also download all project files from the **Course Files** tab via the project files or downloadable PDF summary.

Tensorflow Tutorial.ipynb

Found in the Project root folder

This code builds a simple linear regression model using TensorFlow. It begins by generating a random dataset, then constructs a Model class to predict outputs based on inputs and adjustable parameters (weight and bias). The model is trained over a number of iterations (epochs), adjusting the parameters to minimize loss. Finally, the model's output is visualized overlaid on the original data.

```
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np

m = 2
b = 0.5
x = np.linspace(0,4,100)
y = m * x + b + np.random.randn(*x.shape) + 0.25

plt.scatter(x,y)

class Model:
    def __init__(self):
        self.weight = tf.Variable(10.0)
        self.bias = tf.Variable(10.0)

    def __call__(self, x):
        return self.weight * x + self.bias

# model = Model()
# model(5.0)
#     self.weight.assign_sub(15.0)

def calculate_loss(y_actual, y_output):
    return tf.reduce_mean(tf.square(y_actual - y_output))

def train(model, x, y, learning_rate):
    with tf.GradientTape() as gt:
        y_output = model(x)
        loss = calculate_loss(y, y_output)

    new_weight, new_bias = gt.gradient(loss, [model.weight, model.bias])
    model.weight.assign_sub(new_weight * learning_rate)
    model.bias.assign_sub(new_bias * learning_rate)

model = Model()
epochs = 100
learning_rate = 0.15
```

```
for epoch in range(epochs):
    y_output = model(x)
    loss = calculate_loss(y, y_output)
    print(f"Epoch: {epoch}, loss: {loss.numpy()}")
    train(model, x, y, learning_rate)

print(model.weight.numpy())
print(model.bias.numpy())

new_x = np.linspace(0,4,50)
new_y = model.weight.numpy() * new_x + model.bias.numpy()
plt.scatter(new_x,new_y)
plt.scatter(x,y)
```